D5 – Retrospective & Self-Assessment (Part 2)

Recipix v4.1 – CSE 341 Semester Project

Berk Hakan Öge (210104004132)

23 May 2026

1. Retrospective (½ page)

What I would change if I started over. I would lock the Part-2 runtime contract on day one of the project, not on day one of Part 2. The `tokens.py` agreement was what made Part 1 work as parallel code with İsmail, and the equivalent for Part 2 – the runtime contract document with the in-place AST annotation rule, the runtime value dataclasses, the action-verb signature table, the environment chain, and the agreed-upon expected outputs for samples 1 and 2 – should have been an explicit deliverable I drafted alongside the v4.1 spec. Instead I wrote the spec in early May, started Part 2 work on May 20, and had to bolt the contract on under deadline pressure. The Rev 3 form of `part2_plan.md` works, but it took three drafting rounds to converge there, two of which were sharpening it after the developer agent had already started executing.

I would also run Experiment E1 in Part 1, not in Part 2 as a catch-up entry. The catch-up framing is defensible (entry 17 of my D4 documents the omission honestly), but the value of E1 is the comparison between a cold AI grammar and a locked spec, and that comparison would have been more useful before I locked v4.1 than after – it might have surfaced the `quantity_of`-as-parens-free issue or the unit-set drift before the parser was already written.

What did not work and had to be scoped out of v1. Recipe composition (decision #26) was removed cleanly when I realized the alternative was building closures and a nested-environment evaluator that would have doubled the interpreter complexity. User-defined record types are similarly absent – the type-name set is closed in v1 (decision #32), which trades extensibility for a sealed type system that the checker can prove exhaustive. Per-verb action signatures got collapsed to a three-class table (plan §2.3) after I realized twelve bespoke signatures were ∼12 checker branches and a memorization tax for the exam; the cost is that `pour(flour: Mass)` type-checks even though "pour" linguistically implies liquid, and I would address this as a v2 backwards-compatible refinement rather than re-inflate the table now.

The third scoped-out item is more uncomfortable: the type checker does not flag negative quantities in v1 (spec §3, line on unary minus – interpreter accepts them, type checker does not). This is a real soundness gap that would matter as soon as a recipe scales by a negative factor, but I judged that adding a sign discipline to the dimensional type system was too much work to land safely in the final hours.

Hardest design decision. The action-verb three-class collapse (plan §2.3, locking decision #29's homogeneous-list carve-out). It surfaced the underlying design tension between cooking-faithful semantics ("pour" should require a liquid, "blend" should require same-state ingredients) and type-system parsimony (one rule per class beats twelve rules per verb). I tested the decision twice through AI experiments – once in E3 where Opus 4.7 reproduced the twelve-row mistake unprompted, once in the PM review of the interpreter commits where the table was probed against the §10 error list – and both confirmed that the three-class form is the defensible call. But it is still uncomfortable: a Recipix user who reads "pour" in a recipe and

finds it accepts a Mass quantity is going to feel the language doesn't take itself seriously. I made the call because I have to defend it on the exam in a week and "twelve places to be wrong" is harder to defend than "three places to be wrong"; v2 can re-impose per-verb dimensions as an optional refinement without breaking v1 programs.

---

2. Self-Assessment (½ page, six rubric dimensions per handout §5)

Correctness – A. All three sample programs are wired end-to-end: samples 1 and 2 produce rendered cookbook-style output, sample 3 fails at type-check with a clean dimension-mismatch error message. The 152-test suite passes fully – zero failures. Sample 1 demonstrates `scale` correctly multiplying Mass / Volume / Count ingredients, rebinding the `servings` parameter, and re-executing the step body so the `repeat servings times` loop reflects the scaled value (six pour/flip cycles for a 4 → 6 servings scale). Sample 2 demonstrates `substitute` consuming the replacement-ingredient binding so the substituted slot reads cleanly; the non-substituted variant shows the pre-declared replacement alongside the original, which matches the no-`this` design (decision #25).

Design Justification – A−. Every Part-2 contract item is tied to a written rationale: the three-class action-verb collapse (plan §2.3 + decision #29), the in-place AST annotation (plan §2.1), banker's rounding for scaled servings (plan §2.6 + D1 §4.4), the implicit Ingredient → Quantity projection (decision #31 + D1 §4.8), and the type-equivalence split (decision #10). The minor weakness is that `scale` skipping Temperature and Duration is justified in plan §4 task 10 with the cooking-physics argument (intensive vs additive quantities) but isn't tied to a specific Sebesta hook in the rationale; the professor-agent review (D4 entry 22) flagged this as a polish-if-time item.

Use of Sebesta Framework – B. The major framework moves are correctly cited: §6.14 for the structural-vs-name equivalence split, §9.5 for the parameter-passing-as-by-value-because-no-mutation argument, §7.2 for the precedence and associativity table, §7.6 for the left-to-right operand evaluation order. Two places I could tighten: the §3 quantity-arithmetic table doesn't cite Sebesta §6.2 on derived/dimensional types for the $Q/Q \rightarrow$ unitless rule, and the spec's by-value paragraph (§6, decision #16) doesn't explicitly name all four parameter modes from §9.5 (by-value, by-reference, by-result, by-value-result) inline. Both are one-sentence fixes I would land if I had another hour.

Originality – A−. Recipix's design moves are domain-specific and defensible against alternatives: the dimensional type system (decision #3), Pinch as its own primitive rather than `Quantity<Pinch>` (decision #4), `quantity_of` as a unary operator rather than a function because its return type depends on the operand's quantity-field dimension (decision #34), bare-IDENT substitute slots that make decision #28's "ingredient identity is the binding" visible at grammar level (decision #35). E1, E2, and E3 all confirmed that these decisions are not the textbook defaults – the cold AI grammars and recommendations missed each of them.

Code Quality – A−. The Part-2 modules are cleanly separated by responsibility: `runtime_values.py` is pure data with one `normalize_to_base` function, `environment.py` is type-agnostic shared infrastructure used by both checker and interpreter, `interpreter.py` and `rendering.py` are Berk-owned and don't touch the type checker, `typechecker.py` is İsmail-owned and doesn't touch the interpreter. No Part-1 contract was modified. One acknowledged docstring drift survives into the submission: `runtime_values.py:137` describes the `RtRecipe.steps` field as

if it were used for lazy step rendering, but the interpreter eagerly evaluates step bodies at instantiation and `scale` then re-executes them under the rebound `servings`. I left the docstring as-is rather than risk a late-cycle code change that would have rippled through D2/D3 in the final hours; it is filed as a v2 polish item.

Exam Performance (estimate) – A−. I can defend the interpreter mechanics line-by-line: how `scale` re-executes step bodies under a rebound `servings` parameter (D1 §4.4) and why that is consistent with referential transparency (decision #25) – fresh recipe value, no mutation; the banker's rounding for scaled servings (`int(round(s * by))`) and its trade-off against truncation and ceiling; the implicit Ingredient → Quantity projection at every arithmetic call site (decision #31); the loud-raise behavior on `AmbiguousCall` and why that matters for integration testing. I can defend the precedence ladder, the non-associative comparison rule, the left-to-right operand evaluation order in absence of side effects, and the assignment-as-statement choice. The one area I am less rehearsed is articulating spec §10 error #19 (single-return enforcement) as a spec-vs-plan reconciliation question – addressable with a few minutes of review before the 28 May exam.